UNIVERSIDADE PAULISTA (UNIP)

CURSO DE CIÊNCIA DA COMPUTAÇÃO

PROJETO E DESENVOLVIMENTO II

ESTAÇÃO METEOROLÓGICA COM ESP32

Documentação técnica do projeto

MATHEUS BIFARONI CARVALHO

FRANCISCO IL GUK FERMONE CASTRO SOUZA

GUSTAVO PEREIRA BARRETO

LUCAS VICENTE PEREIRA SILVA

Professor: Marco Gomes

SÃO PAULO – SP

2026

MATHEUS BIFARONI CARVALHO

FRANCISCO IL GUK FERMONE CASTRO SOUZA

GUSTAVO PEREIRA BARRETO

LUCAS VICENTE PEREIRA SILVA

ESTAÇÃO METEOROLÓGICA COM ESP32

Documentação técnica apresentada à Universidade Paulista – UNIP, como requisito parcial para avaliação da disciplina Projeto e Desenvolvimento II, do curso de Ciência da Computação.

Professor: Marco Gomes

SÃO PAULO – SP

2026

RESUMO

Este trabalho apresenta o desenvolvimento de uma estação meteorológica de baixo custo baseada em ESP32, concebida para aquisição e disponibilização de dados ambientais em uma interface web. A proposta foi desenvolvida no contexto da disciplina Projeto e Desenvolvimento II, integrando conhecimentos de Internet das Coisas, sistemas embarcados, sensores, circuitos digitais, programação em Python, banco de dados e desenvolvimento web. O protótipo final utiliza os sensores DHT22, BMP280, MQ-135 e LDR, responsáveis respectivamente pela obtenção de umidade e temperatura, pressão atmosférica e temperatura, indicação de qualidade do ar e luminosidade. O pluviômetro previsto na proposta inicial foi posteriormente retirado do escopo pelo docente em razão do custo do componente, não fazendo parte da versão final. A implementação do hardware foi realizada sobre uma protoboard e controlada por um ESP32 DEVKIT V1, com firmware em MicroPython. O sistema realiza leituras periódicas dos sensores e disponibiliza os dados por HTTP, permitindo que o dashboard web consuma as informações em tempo real. O conjunto de software inclui ainda componentes de backend e persistência em banco de dados, estruturados em Python e SQL, além do dashboard desenvolvido com HTML, CSS e JavaScript. Durante os testes, os componentes foram inicialmente validados de forma individual e, posteriormente, integrados em um único protótipo. Os resultados registrados para a etapa de entrega mostraram funcionamento do DHT22, LDR, MQ-135 e BMP280, bem como a integração do protótipo com a interface web por meio de conexão Wi-Fi. No teste integrado, os dados apresentados no dashboard foram provenientes diretamente das leituras realizadas pelo ESP32, com atualização visual observada em aproximadamente dois segundos e sem ocorrência de travamentos ou perda de conexão durante a sessão registrada. A experiência também permitiu identificar limitações práticas do protótipo, especialmente quanto ao condicionamento de sinais analógicos, à dependência da rede local e à necessidade de futuras melhorias de calibração e encapsulamento. O projeto demonstra, em escala acadêmica, como sensores, processamento embarcado e interfaces digitais podem ser combinados para produzir informações ambientais acessíveis ao usuário.

Palavras-chave: Cidades Inteligentes; ESP32; Internet das Coisas; MicroPython; Estação Meteorológica; Dashboard Web; Sensoriamento Ambiental.

ABSTRACT

This work presents the development of a low-cost weather station based on ESP32, designed to acquire and provide environmental data through a web interface. The project was developed in the context of the Project and Development II course, integrating knowledge of Internet of Things, embedded systems, sensors, digital circuits, Python programming, databases, and web development. The final prototype uses DHT22, BMP280, MQ-135, and LDR sensors, responsible respectively for measuring humidity and temperature, atmospheric pressure and temperature, indicating air quality, and measuring light. The rain gauge originally planned was later removed from the scope by the instructor because of the component cost and is therefore not part of the final version. The hardware was assembled on a breadboard and controlled by an ESP32 DEVKIT V1, with firmware written in MicroPython. The system performs periodic sensor readings and provides the data over HTTP, allowing the web dashboard to consume the information in real time. The software set also includes backend and database persistence components structured in Python and SQL, in addition to a dashboard developed with HTML, CSS, and JavaScript. During testing, the components were first validated individually and then integrated into a single prototype. The final delivery tests showed operation of the DHT22, LDR, MQ-135, and BMP280, as well as integration of the prototype with the web interface through Wi-Fi. In the integrated test, the dashboard values came directly from readings performed by the ESP32, with visual updates observed at approximately two-second intervals and without freezes or connection loss during the recorded session. The experience also identified practical limitations, especially concerning analog signal conditioning, dependence on the local network, and the need for future calibration and enclosure improvements. The project demonstrates, on an academic prototype scale, how sensors, embedded processing, and digital interfaces can be combined to produce accessible environmental information.

Keywords: Smart Cities; ESP32; Internet of Things; MicroPython; Weather Station; Web Dashboard; Environmental Sensing.

LISTA DE FIGURAS

Figura 1 – Protótipo final da estação meteorológica montado em protoboard.....14
Figura 2 – Dashboard final exibindo dados do protótipo em funcionamento.....15
Figura 3 – Teste isolado do BMP280 com leitura de temperatura e pressão no ambiente de desenvolvimento.....16
Figura 4 – Registros distintos do dashboard durante a etapa final, evidenciando alteração do valor de luminosidade.....17

LISTA DE TABELAS

Tabela 1 – Divisão de responsabilidades da equipe.....6
Tabela 2 – Histórico de desenvolvimento do projeto.....7
Tabela 3 – Requisitos funcionais e não funcionais.....8
Tabela 4 – Pinagem da estação meteorológica.....9
Tabela 5 – Sensores e componentes utilizados.....9
Tabela 6 – Estrutura de dados da tabela estacao.....11
Tabela 7 – Estrutura de dados da tabela leitura.....11
Tabela 8 – Condições e etapas dos testes.....12
Tabela 9 – Resultados dos testes realizados.....13
Tabela 10 – Tabela de custos (BOM).....50

LISTA DE ABREVIATURAS E SIGLAS

Sigla	**Significado**
ADC	Conversor Analógico-Digital
API	Application Programming Interface
BMP	Barometric Pressure Sensor (BMP280)
ESP32	Microcontrolador/módulo baseado no SoC ESP32
GPIO	General Purpose Input/Output
HTTP	Hypertext Transfer Protocol
I²C	Inter-Integrated Circuit
IoT	Internet of Things – Internet das Coisas
LDR	Light Dependent Resistor
MQ	Família de sensores semicondutores para gases
PPM	Partes por milhão
RF	Requisito Funcional
RNF	Requisito Não Funcional
UR	Umidade Relativa

SUMÁRIO

1 INTRODUÇÃO.....1
1.1 Contextualização e Justificativa.....1
1.2 Objetivo Geral.....1
1.3 Objetivos Específicos.....1
1.4 Definição de Termos-Chave.....2
2 REVISÃO BIBLIOGRÁFICA (FUNDAMENTAÇÃO TEÓRICA).....3
2.1 Cidades Inteligentes e Internet das Coisas.....3
2.2 ESP32 e MicroPython.....3
2.3 Sensores e Sinais.....3
2.4 DHT22.....4
2.5 BMP280.....4
2.6 MQ-135.....4
2.7 LDR.....4
2.8 Pluviômetro e alteração de escopo.....5
2.9 Circuitos Digitais e Lógica Matemática.....5
2.10 Programação, Backend e Banco de Dados.....5
2.11 Geometria Analítica e Localização.....5
3 METODOLOGIA E DESENVOLVIMENTO.....6
3.1 Gestão do Projeto e Metodologia de Trabalho.....6
3.2 Histórico de Desenvolvimento.....7
3.3 Escopo e Alteração do Pluviômetro.....7
3.4 Requisitos Funcionais e Não Funcionais.....8
3.5 Arquitetura Geral do Sistema.....8
3.6 Projeto de Hardware e Circuitos Digitais.....9
3.7 Tabela de Sensores e Componentes.....9
3.8 Desenvolvimento do Firmware (ESP32).....10
3.9 Backend em Python e Regras de Negócio.....10
3.10 Modelagem do Banco de Dados.....11
3.11 Frontend e Dashboard Público.....11
3.12 Plano de Testes e Cronograma.....12
4 RESULTADOS.....13
4.1 Protótipo Físico Final.....14
4.2 Dashboard em Funcionamento.....15
4.3 Teste Isolado do BMP280.....16
4.4 Variação de Luminosidade.....17
5 ANÁLISE DOS RESULTADOS E DISCUSSÃO.....19
5.1 Confiabilidade e Precisão do Hardware.....19
5.2 Eficácia dos Filtros Matemáticos.....19
5.3 Desempenho da Transmissão e Arquitetura.....19
5.4 Usabilidade e Geolocalização.....20

5.5 Aderência ao Conceito de Smart Cities....20
5.6 Anomalias, Ajustes e Limitações....20
6 CONCLUSÕES E RECOMENDAÇÕES....21
6.1 Conclusão....21
6.2 Recomendações e Trabalhos Futuros....21
REFERÊNCIAS....23
APÊNDICES....24
Apêndice A – Firmware do ESP32....25
Apêndice B – API / Backend em Python....33
Apêndice C – Dashboard Web....37
Apêndice D – Script SQL do Banco de Dados....49
Apêndice E – Tabela de Custos (BOM)....50
DECLARAÇÃO DE USO DE INTELIGÊNCIA ARTIFICIAL....51

1 INTRODUÇÃO

1.1 Contextualização e Justificativa

O desenvolvimento de Cidades Inteligentes depende da combinação entre infraestrutura física, sistemas computacionais e tecnologias de comunicação capazes de produzir e disponibilizar informações úteis à sociedade. No contexto da Internet das Coisas (IoT), dispositivos físicos podem ser conectados a redes e sistemas de informação para realizar aquisição, processamento e comunicação de dados, permitindo o desenvolvimento de aplicações distribuídas em diferentes domínios (ITU-T, 2012).

No ambiente urbano, o monitoramento de condições ambientais pode apoiar a compreensão de fenômenos locais e facilitar o acesso público a informações relacionadas a temperatura, umidade, pressão atmosférica, luminosidade e qualidade do ar. Uma estação meteorológica de baixo custo constitui, nesse sentido, uma alternativa adequada para fins de prototipagem acadêmica, pois permite explorar de forma integrada sensores físicos, microcontroladores, comunicação de dados, armazenamento e interfaces web.

A proposta deste projeto justifica-se pela possibilidade de construir uma cadeia tecnológica completa em pequena escala, aproximando conteúdos de diferentes áreas da formação em Ciência da Computação. Além do caráter técnico, a disponibilização de dados em uma interface web favorece a visualização das medições e a compreensão do funcionamento de um sistema de monitoramento ambiental orientado à informação pública.

1.2 Objetivo Geral

Desenvolver e validar um sistema embarcado de monitoramento ambiental, na forma de uma estação meteorológica baseada em ESP32, integrando sensores físicos, firmware em MicroPython e uma interface web capaz de apresentar dados captados pelo protótipo em tempo real.

1.3 Objetivos Específicos

- Projetar e montar o circuito eletrônico utilizando ESP32, DHT22, BMP280, MQ-135 e LDR.

• Desenvolver o firmware do ESP32 para leitura, tratamento e disponibilização dos dados dos sensores.

• Implementar componentes de software em Python para recepção, validação e persistência das leituras.

• Modelar a estrutura relacional necessária para o armazenamento das medições.

• Criar uma interface web responsiva para visualização das variáveis ambientais.

• Aplicar conceitos de circuitos digitais, programação, lógica matemática, banco de dados e geometria analítica ao desenvolvimento do projeto.

• Validar individualmente os sensores e, posteriormente, o funcionamento integrado do protótipo.

1.4 Definição de Termos-Chave

Termo	Definição no contexto do projeto
Firmware	Software executado diretamente no ESP32 para inicialização dos periféricos, leitura dos sensores e comunicação.
Telemetria	Transmissão de dados medidos pelo protótipo para outro sistema computacional.
MicroPython	Implementação da linguagem Python voltada a microcontroladores, utilizada no firmware do projeto.
API	Interface programável utilizada para receber ou disponibilizar dados entre componentes de software.
Payload	Conjunto estruturado de dados transportado em uma requisição ou mensagem.
Geolocalização	Representação da posição da estação por meio de coordenadas geográficas.

2 REVISÃO BIBLIOGRÁFICA (FUNDAMENTAÇÃO TEÓRICA)

2.1 Cidades Inteligentes e Internet das Coisas

A Internet das Coisas pode ser compreendida como uma infraestrutura que conecta objetos físicos e virtuais por meio de tecnologias de informação e comunicação, permitindo capacidades de identificação, comunicação, aquisição e processamento de dados (ITU-T, 2012). Essa característica torna a IoT especialmente aplicável ao monitoramento ambiental, em que dispositivos sensores produzem medições contínuas e sistemas computacionais transformam essas medições em informação disponível ao usuário.

No presente projeto, a estação meteorológica representa um sistema IoT em escala de protótipo: os sensores constituem a camada de aquisição, o ESP32 executa o processamento local e a comunicação sem fio, e a camada de software apresenta e organiza as leituras. A solução não pretende reproduzir uma infraestrutura meteorológica profissional, mas demonstrar a integração dos principais componentes de uma aplicação conectada.

2.2 ESP32 e MicroPython

O ESP32 reúne processamento, memória, interfaces de entrada e saída e conectividade sem fio em uma plataforma adequada a aplicações embarcadas e IoT. O material técnico disponibilizado na disciplina também descreve o suporte a Wi-Fi, Bluetooth, GPIO, ADC e barramentos seriais, recursos empregados na integração dos sensores. Kolban (2018) destaca a flexibilidade do ESP32 para aplicações que combinam periféricos, processamento e comunicação.

Para o desenvolvimento do firmware, foi utilizado MicroPython. O material da disciplina recomenda o uso do Thonny IDE para escrever e enviar scripts ao ESP32, ambiente efetivamente utilizado nos testes individuais realizados pela equipe. A escolha favorece a prototipagem rápida e permite acesso direto aos periféricos do microcontrolador.

2.3 Sensores e Sinais

Sensores transformam variações de grandezas físicas em sinais passíveis de leitura por sistemas de controle. Wendling (2010) distingue o comportamento de sensores analógicos e

digitais, enquanto Thomazini e Pedro (2020) descrevem a importância da interpretação dos sinais e de seu condicionamento conforme a aplicação. No projeto, o DHT22 utiliza sinal digital, enquanto LDR e MQ-135 fornecem leituras analógicas ao ADC do ESP32, e o BMP280 utiliza o barramento I²C.

2.4 DHT22

O DHT22, também associado ao AM2302, é utilizado para medição de temperatura e umidade relativa do ar. O material técnico da disciplina descreve o dispositivo como um sensor digital que transmite os dados em uma trama serial própria. Durante os testes do projeto, o componente foi conectado ao GPIO D4 e apresentou leitura de 24 °C e 66,9% de umidade relativa, valor arredondado a partir do registro observado no vídeo.

2.5 BMP280

O BMP280 é um sensor de pressão barométrica e temperatura com interface I²C ou SPI. O material técnico destaca que o dispositivo possui coeficientes de calibração de fábrica e realiza processamento interno para compensação das leituras. No projeto, a comunicação foi realizada por I²C utilizando SDA no GPIO 21 e SCL no GPIO 22.

2.6 MQ-135

O MQ-135 é um sensor quimiorresistivo utilizado como indicador da presença de determinados gases e compostos no ambiente. Sua leitura é analógica e depende de condições de alimentação, aquecimento e calibração. No firmware final analisado, a equipe incorporou média de múltiplas amostras e uma referência de baseline para produzir uma indicação em PPM. Por se tratar de um protótipo de baixo custo, os valores apresentados devem ser interpretados no contexto da calibração e do circuito efetivamente utilizado.

2.7 LDR

O LDR é um fotoresistor cuja resistência elétrica varia com a incidência de luz. Guadagnini, Rocha e Barlette (2019) discutem aplicações didáticas de sensores de luminosidade e sua aquisição automática. No projeto, o LDR foi ligado ao ADC do ESP32 e utilizado para estimar a iluminância em uma escala apresentada pelo dashboard em lux. O comportamento

observado durante o teste integrado mostrou variação do valor de luminosidade conforme a condição de iluminação do ambiente.

2.8 Pluviômetro e alteração de escopo

O pluviômetro de báscula transforma a precipitação captada em eventos de basculamento e pulsos elétricos, permitindo estimar a lâmina de chuva acumulada. Embora o componente estivesse presente na proposta inicial do projeto, ele foi retirado do escopo por orientação do docente em razão do custo de aquisição. Assim, o protótipo final e a documentação desta entrega consideram apenas DHT22, BMP280, MQ-135 e LDR.

2.9 Circuitos Digitais e Lógica Matemática

A leitura dos sensores exige a distinção entre sinais digitais, analógicos e protocolos seriais. O DHT22 é interpretado por uma entrada digital, o LDR e o MQ-135 utilizam ADC e o BMP280 opera por I²C. Conceitos de lógica digital e validação matemática aparecem ainda nas rotinas condicionais que classificam faixas de leitura e nos procedimentos de tratamento de dados.

2.10 Programação, Backend e Banco de Dados

O backend em Python é responsável, no conjunto de artefatos do projeto, pela estruturação da comunicação de dados e pela persistência relacional. O material do grupo inclui uma API FastAPI e um script SQL com as tabelas de estação e leitura. A modelagem relacional permite associar cada conjunto de medições a uma estação identificada por coordenadas geográficas.

2.11 Geometria Analítica e Localização

A representação da posição da estação utiliza coordenadas de latitude e longitude. O conjunto de software registra a posição da estação em São Paulo nas coordenadas -23,55052 e -46,63330. Esses valores permitem representar o ponto espacialmente e constituem a base para recursos futuros de mapa e cálculo de distância.

3 METODOLOGIA E DESENVOLVIMENTO

3.1 Gestão do Projeto e Metodologia de Trabalho

A equipe foi formada por quatro integrantes e organizou o desenvolvimento por divisão de responsabilidades e reuniões presenciais. Não foram identificados registros formais de um framework Scrum completo, com backlog, Product Owner e sprints documentadas; a dinâmica efetivamente observada foi incremental, com distribuição de funções, desenvolvimento paralelo, sessões de teste e correção de problemas.

Tabela 1 – Divisão de responsabilidades da equipe.

Integrante	Responsabilidade
Lucas Vicente Pereira Silva	Front-end do site.
Gustavo Pereira Barreto	Back-end e estrutura de software/banco de dados.
Matheus Bifaroni Carvalho	Documentação técnica e organização do relatório.
Francisco Il Guk Fermone Castro Souza	Documentação técnica em conjunto com Matheus; apoio aos testes e integração.
Todos os integrantes	Montagem do protótipo, simulações, testes e integração presencial.

Fonte: elaborado pelos autores (2026).

3.2 Histórico de Desenvolvimento

Tabela 2 – Histórico de desenvolvimento do projeto.

Data	Atividade
24/08/2026	Apresentação do projeto em aula e aquisição dos componentes.
22/09/2026	Primeira reunião autônoma: primeira tentativa de simulação no Wokwi; não houve teste efetivo por falta de familiaridade com componentes e ferramenta.
24/09/2026	Segunda reunião autônoma com orientação do professor Ricardo sobre a metodologia de teste individual dos componentes e atenção ao contato da protoboard.
25/09/2026	Terceira reunião autônoma: primeiros testes individuais com DHT22, LDR e MQ-135, usando Thonny.
28/09/2026	Quarta reunião autônoma: continuidade da montagem, ajustes e coleta de informações.
29/09/2026	Quinta reunião autônoma: continuidade da integração do hardware e software.
30/09/2026	Sexta reunião autônoma: teste isolado do BMP280 e integração do protótipo completo com dashboard via Wi-Fi.
01/10/2026	Orientação adicional do professor Ricardo para melhoria futura do condicionamento do LDR e MQ-135; docente considerou os dados adequados para apresentação e documentação.
02/10/2026	Atualização final do software do dashboard e consolidação dos artefatos para a entrega.

Fonte: elaborado pelos autores (2026).

3.3 Escopo e Alteração do Pluviômetro

O escopo final compreendeu a aquisição e leitura de temperatura, umidade relativa, pressão atmosférica, qualidade do ar e luminosidade; o envio ou disponibilização dessas informações; a exibição por interface web; e a estruturação de software para persistência em banco de dados. O pluviômetro foi excluído por decisão do docente e, portanto, os dados de precipitação não fazem parte do protótipo final.

3.4 Requisitos Funcionais e Não Funcionais

Tabela 3 – Requisitos funcionais e não funcionais.

ID	Tipo	Requisito	Situação na entrega
RF01	Funcional	O ESP32 deve efetuar a leitura dos sensores em intervalos programados.	Atendido; o firmware final executa ciclos periódicos de leitura.
RF02	Funcional	O sistema deve disponibilizar os dados por comunicação HTTP.	Atendido no protótipo demonstrado por endpoint HTTP do ESP32.
RF03	Funcional	A posição da estação deve ser representável por coordenadas geográficas.	Parcialmente atendido; coordenadas reais são registradas, enquanto a interface final capturada não exibe o componente de mapa.
RF04	Funcional	O dashboard deve emitir alertas visuais para condições ambientais críticas.	Atendido/adaptado para qualidade do ar e temperatura, sem precipitação após retirada do pluviômetro.
RNF01	Não funcional	A interface deve se adaptar a diferentes tamanhos de tela.	Atendido por layout baseado em grid responsivo e elementos com auto-fit.
RNF02	Não funcional	O tempo de atualização do dashboard não deve exceder 5 s.	Atendido; o dashboard realiza polling a cada 2 s no código final.
RNF03	Não funcional	O backend deve validar valores fora de faixas aceitáveis.	Parcial; há validação estrutural/tipagem em artefatos de API, mas o fluxo de demonstração final é direto entre ESP32 e dashboard.

Fonte: elaborado pelos autores (2026).

3.5 Arquitetura Geral do Sistema

A arquitetura efetivamente demonstrada na sessão final utiliza o ESP32 como núcleo de aquisição e também como servidor HTTP local do protótipo. Os sensores são lidos pelo firmware em MicroPython; o endpoint /api/dados disponibiliza um objeto JSON; e o dashboard, servido como aplicação web, consulta esse endpoint por meio de requisições periódicas. O pacote de software também contém uma implementação separada de API em FastAPI e scripts de persistência em banco de dados, utilizados como artefatos de desenvolvimento e modelagem da camada de software.

Sensores	→	**ESP32 MicroPython**	→	**HTTP /api/dados**	→	**Dashboard Web**

A persistência relacional está estruturada separadamente por meio de uma API FastAPI e um esquema SQL. Essa camada encontra-se documentada nos apêndices porque faz parte dos artefatos produzidos pelo grupo, mesmo que a demonstração final registrada em vídeo tenha utilizado diretamente o endpoint HTTP do ESP32.

3.6 Projeto de Hardware e Circuitos Digitais

O protótipo foi montado em protoboard e utiliza um ESP32 DEVKIT V1. O DHT22 foi conectado ao pino D4 e fornece sinal digital. O LDR foi conectado ao ADC do GPIO D34 e o MQ-135 ao ADC do GPIO D35. O BMP280 utiliza I²C, com SDA no GPIO D21 e SCL no GPIO D22. A equipe informou que não foram utilizados resistores externos nem transistores na configuração testada. Durante a orientação de 01/10, o professor Ricardo sugeriu que ajustes adicionais de condicionamento poderiam ser feitos futuramente, mas considerou a configuração adequada para a entrega.

Tabela 4 – Pinagem da estação meteorológica.

Componente	VCC	GND	Sinal / GPIO	Tipo de sinal
DHT22	Positivo	Negativo	DATA = D4	Digital
LDR	Positivo	Negativo	AO = D34	Analógico
MQ-135	Positivo	Negativo	AO = D35	Analógico
BMP280	Positivo	Negativo	SDA = D21; SCL = D22	Digital / I²C

Fonte: elaborado pelos autores (2026).

3.7 Tabela de Sensores e Componentes

Tabela 5 – Sensores e componentes utilizados.

Componente	Modelo/módulo informado	Função
ESP32	DEVKIT V1	Controle do protótipo, aquisição e comunicação.
DHT22	AM2302 Arduino / AM2302	Temperatura e umidade relativa.
LDR	Arduino Node Rasp	Luminosidade por leitura analógica.
MQ-135	Modelo genérico informado pelo grupo	Indicação de qualidade do ar/gases.
BMP280	Sensor de Pressão Bmp280 Barométrico Para Arduino	Pressão atmosférica e temperatura.

Fonte: elaborado pelos autores (2026).

3.8 Desenvolvimento do Firmware (ESP32)

O firmware final do pacote utiliza MicroPython e inicializa o barramento I²C do BMP280, o DHT22 e as entradas ADC do LDR e do MQ-135. A rotina do BMP280 realiza leitura dos registradores de calibração e compensação matemática para temperatura e pressão. Para o MQ-135, a versão mais recente do código incorpora oito amostras para média, uma referência de baseline R0 e uma função de transformação para indicação em PPM. Para o LDR, o firmware utiliza o ADC e uma equação aproximada para estimar luminosidade.

A comunicação demonstrada utiliza HTTP. O ESP32 também implementa um pequeno servidor assíncrono para disponibilizar os arquivos do dashboard e a rota /api/dados. O monitor de Wi-Fi verifica o estado da conexão periodicamente e tenta reconectar quando o sinal não está disponível. O código final registra um novo ciclo de leitura e disponibilização a cada cinco segundos, enquanto a interface realiza consultas aproximadamente a cada dois segundos; assim, a atualização visual pode ocorrer mais rapidamente que a produção de uma nova amostra do firmware.

3.9 Backend em Python e Regras de Negócio

O conjunto de software contém uma API desenvolvida com FastAPI e Pydantic. O arquivo de backend apresentado no pacote inclui um endpoint POST para receber telemetria e uma rotina de consulta às leituras recentes, além de integração com banco de dados relacional. Há também uma variante integrada com SQLite na pasta estacao_meteorologica_integrada. Para esta documentação, o backend é tratado como camada de software e modelagem de persistência desenvolvida pelo grupo, enquanto o teste final demonstrado foi realizado diretamente pelo endpoint HTTP do ESP32.

No firmware, regras matemáticas aparecem no cálculo de pressão e temperatura do BMP280, na média de oito amostras do MQ-135 e na transformação da leitura analógica do LDR. O tratamento do MQ-135 merece destaque porque a versão final substituiu uma leitura direta por média de amostras e uma referência de baseline, reduzindo oscilações instantâneas do ADC antes da transformação do indicador.

3.10 Modelagem do Banco de Dados

O esquema SQL fornecido pelo grupo contém as entidades estacao e leitura. A tabela estacao identifica o ponto monitorado e armazena latitude, longitude e descrição. A tabela leitura registra cada medição, vinculando-a à estação por chave estrangeira e armazenando temperatura, umidade, pressão, qualidade do ar, luminosidade e data/hora. O pluviômetro não aparece no esquema final.

Tabela 6 – Estrutura de dados da tabela estacao.

Tabela	Atributo	Tipo	Restrição / Descrição
estacao	id_estacao	INT	PK, AUTO_INCREMENT
estacao	latitude	FLOAT	NOT NULL; coordenada geográfica
estacao	longitude	FLOAT	NOT NULL; coordenada geográfica
estacao	descricao	VARCHAR(100)	Descrição da estação

Fonte: elaborado pelos autores (2026).

Tabela 7 – Estrutura de dados da tabela leitura.

Tabela	Atributo	Tipo	Restrição / Descrição
leitura	id_leitura	BIGINT	PK, AUTO_INCREMENT
leitura	id_estacao	INT	NOT NULL; FK para estacao(id_estacao)
leitura	temperatura	FLOAT	NOT NULL; °C
leitura	umidade	FLOAT	NOT NULL; %
leitura	pressao	FLOAT	NOT NULL; hPa
leitura	qualidade_ar	INT	NOT NULL; indicador em PPM
leitura	luminosidade	INT	NOT NULL; indicador de luminosidade
leitura	data_hora	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

Fonte: elaborado pelos autores (2026).

3.11 Frontend e Dashboard Público

A interface final analisada utiliza HTML, CSS e JavaScript e apresenta cinco indicadores principais: temperatura, pressão atmosférica, umidade relativa, qualidade do ar e luminosidade. A interface também exibe informações de diagnóstico do hardware e da rede, horário de

sincronização, estado da API e identificação do sensor barométrico. O CSS emprega grid com auto-fit, contribuindo para adaptação da disposição dos componentes a diferentes larguras de tela.

A versão de apresentação do dashboard consulta o endpoint HTTP do ESP32 e atualiza os indicadores aproximadamente a cada dois segundos. Durante o teste integrado, os quatro sensores estavam conectados simultaneamente e os dados exibidos foram confirmados como provenientes das leituras do ESP32. A captura final apresenta a estação online, os valores de temperatura, pressão, umidade, qualidade do ar e luminosidade e informações do barramento I²C.

3.12 Plano de Testes e Cronograma

Tabela 8 – Condições e etapas dos testes.

Etapa	Data/horário	Local	Procedimento
Teste 1 – DHT22, LDR e MQ-135	25/09/2026, 12h30–13h30	Laboratório de informática, 1º andar	Testes individuais via USB/Thonny.
Teste 2 – BMP280	30/09/2026, 12h30–13h30	Laboratório de informática, 1º andar	Teste individual via USB/Thonny.
Teste 3 – Integração final	30/09/2026, 12h30–13h30	Laboratório de informática, 1º andar	Todos os sensores conectados, Wi-Fi roteado pelo Francisco e dashboard ativo.
Orientação complementar	01/10/2026	Aula de Circuitos Lógicos Digitais	Revisão da montagem e discussão sobre condicionamento analógico.

Fonte: elaborado pelos autores (2026).

4 RESULTADOS

Os resultados desta seção são apresentados de forma objetiva, com base nas medições, registros fotográficos, vídeos e comportamento observados pela equipe durante os testes. O protótipo final foi montado e os quatro sensores utilizados foram submetidos a testes individuais e a uma etapa de integração.

Tabela 9 – Resultados dos testes realizados.

ID	Componente / etapa	Procedimento	Resultado observado	Situação
CT01	DHT22	Conexão USB ao notebook do Francisco e teste no Thonny.	24,0 °C e 66,9% de umidade relativa.	Aprovado
CT02	LDR	Conexão USB e leitura no Thonny sob iluminação do ambiente.	Aproximadamente 65 lux, com variação conforme a iluminação.	Aprovado
CT03	MQ-135	Conexão USB e leitura no Thonny.	185 ppm no registro do teste individual.	Aprovado
CT04	BMP280 – teste isolado	Conexão USB e leitura no Thonny.	Aproximadamente 23,1 °C e 927,6–927,8 hPa no registro visual do teste.	Aprovado
CT05	Integração final	Quatro sensores conectados; ESP32 em Wi-Fi e dashboard ativo.	Dados reais exibidos simultaneamente no site; 26,2 °C; 929,5 hPa; 66,9–67,0% UR; 400 ppm; 64,5–75,8 lux em capturas distintas.	Aprovado
CT06	Comunicação integrada	Observação do dashboard durante a sessão final.	Atualização visual em aproximadamente 2 s, sem travamentos ou perda de conexão observados.	Aprovado

Fonte: elaborado pelos autores (2026).

4.1 Protótipo Físico Final

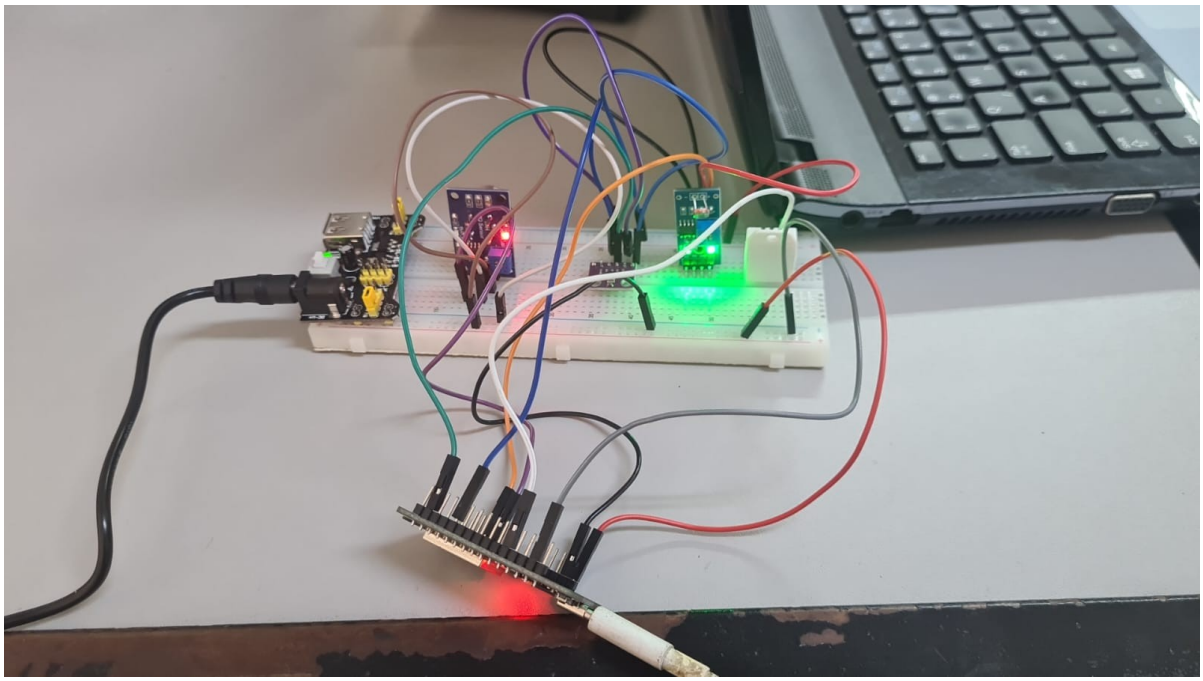


Figura 1 – Protótipo final da estação meteorológica montado em protoboard.
Fonte: acervo da equipe (2026).

A imagem registra o ESP32, a protoboard e os sensores conectados durante a etapa final de integração. A montagem foi alimentada e conectada ao computador utilizado pela equipe durante os testes.

4.2 Dashboard em Funcionamento

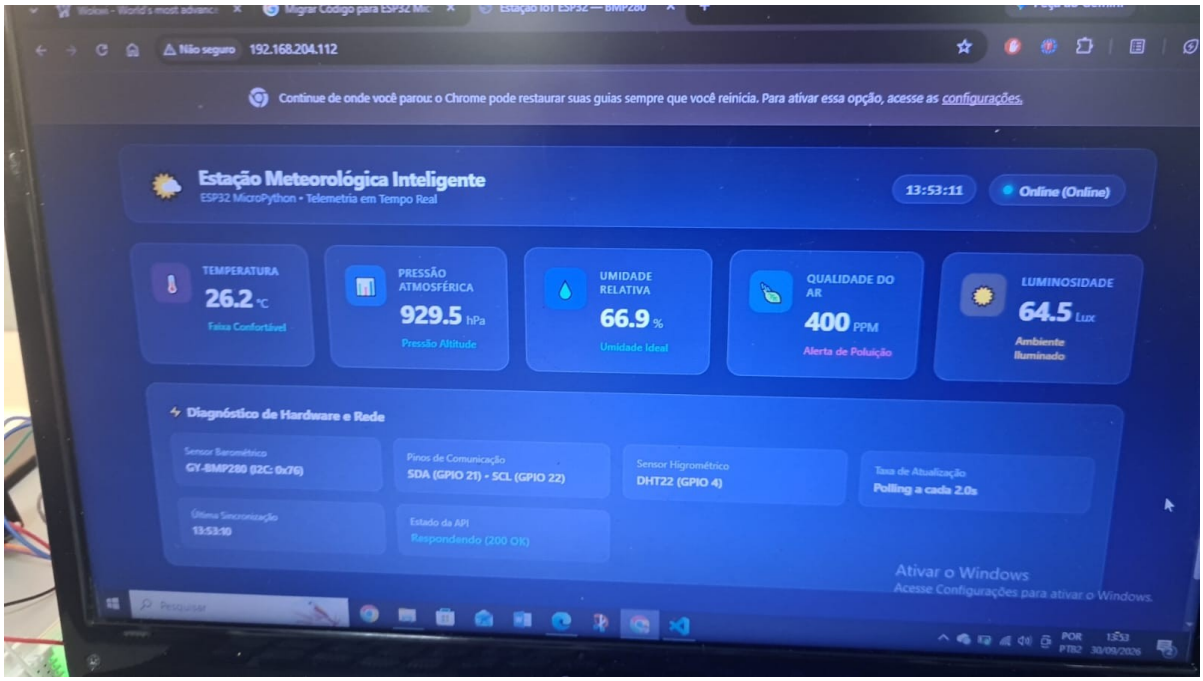


Figura 2 – Dashboard final exibindo dados do protótipo em funcionamento.
Fonte: acervo da equipe (2026).

A captura mostra os cinco indicadores ambientais preenchidos e o estado online do sistema. Também são apresentadas informações de diagnóstico, incluindo identificação do GY-BMP280, os pinos I²C, o DHT22 e a última sincronização.

4.3 Teste Isolado do BMP280

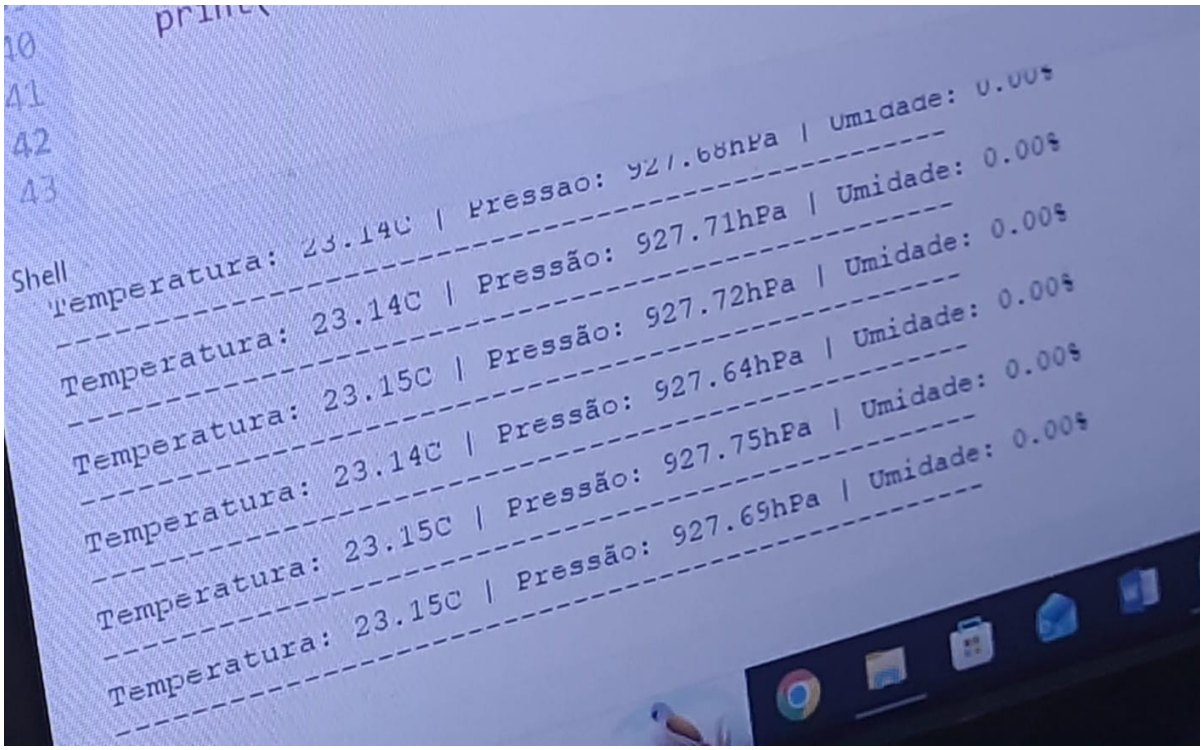


Figura 3 – Teste isolado do BMP280 com leitura de temperatura e pressão no ambiente de desenvolvimento.

Fonte: acervo da equipe (2026).

O teste individual do BMP280 foi realizado no Thonny antes da integração geral. O registro visual apresenta sucessivas leituras próximas de 23,1 °C e 927,6–927,8 hPa, demonstrando a leitura do sensor por I²C e sua variação entre amostras.

4.4 Variação de Luminosidade

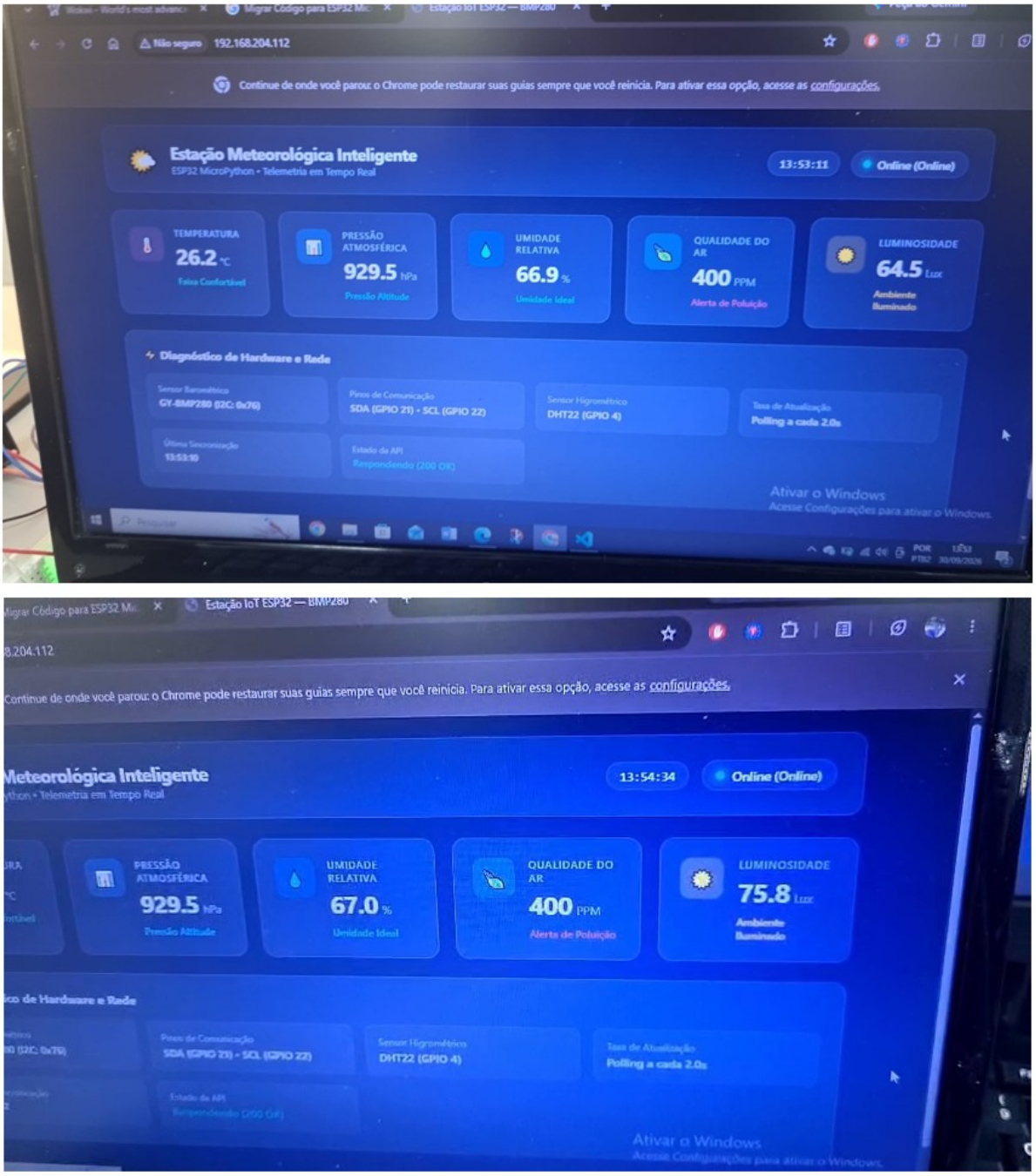


Figura 4 – Registros distintos do dashboard durante a etapa final, evidenciando alteração do valor de luminosidade.

Fonte: acervo da equipe (2026).

Durante o teste integrado, a alteração da iluminação ambiente produziu variações no valor apresentado pelo dashboard, com registros observados de aproximadamente 64,5 lux e 75,8 lux

em momentos distintos. O comportamento foi utilizado pela equipe como evidência de resposta do sensor às condições de iluminação.

5 ANÁLISE DOS RESULTADOS E DISCUSSÃO

5.1 Confiabilidade e Precisão do Hardware

Os testes individuais realizados em 25/09 mostraram resposta funcional do DHT22, LDR e MQ-135, enquanto o BMP280 foi validado individualmente em 30/09. A etapa de integração demonstrou que os quatro sensores podiam permanecer conectados simultaneamente sem interrupção da sessão registrada. A ausência de travamentos e de perda de conexão durante o teste final constitui evidência de estabilidade para a condição controlada de laboratório, embora não represente validação metrológica em longo prazo ou em ambiente externo.

As medições de DHT22 e BMP280 apresentam valores coerentes com o tipo de grandeza monitorada. No caso do LDR e do MQ-135, os valores dependem do circuito, da alimentação e da calibração aplicada. A orientação de 01/10 registrou uma recomendação de melhoria do condicionamento desses sinais; o docente, entretanto, considerou os resultados suficientes para a apresentação e documentação da etapa de entrega.

5.2 Eficácia dos Filtros Matemáticos

O firmware final do MQ-135 passou a utilizar média de oito amostras do ADC e um baseline R0 antes da transformação da leitura em indicador de concentração. A alteração reduz a influência de oscilações instantâneas do conversor e melhora a estabilidade do valor apresentado, mas não equivale, por si só, a uma calibração laboratorial rastreável. No LDR, a conversão para luminosidade é aproximada e adequada ao contexto demonstrativo do protótipo.

5.3 Desempenho da Transmissão e Arquitetura

A transmissão demonstrada no protótipo final ocorreu por Wi-Fi e HTTP, com o ESP32 disponibilizando um endpoint de dados que foi consultado pelo dashboard. Durante a sessão de 30/09, a interface foi atualizada aproximadamente a cada dois segundos e não foram observadas falhas de conexão. O código do firmware, contudo, realiza novos ciclos de aquisição a cada cinco segundos; portanto, o intervalo visual do dashboard e o intervalo de geração de amostras não são idênticos. Essa distinção é relevante para a interpretação do desempenho do sistema.

5.4 Usabilidade e Geolocalização

A interface final apresenta indicadores grandes e estados textuais, facilitando a leitura dos valores pelo usuário. A estrutura CSS baseada em grid com auto-fit permite reorganização dos cards em diferentes larguras. As coordenadas -23,55052 e -46,63330 foram confirmadas como referentes ao local da estação. O pacote de software também contém uma versão integrada com recursos de histórico e mapa; na captura final utilizada como evidência, entretanto, o foco visual está nos cinco indicadores e no diagnóstico do sistema.

5.5 Aderência ao Conceito de Smart Cities

O projeto reproduz, em escala acadêmica, elementos centrais de uma aplicação IoT voltada ao contexto urbano: aquisição de dados por sensores, processamento em dispositivo embarcado, comunicação sem fio e visualização digital. A solução não deve ser interpretada como sistema meteorológico profissional, mas como demonstração de como informações ambientais podem ser coletadas e disponibilizadas de forma acessível para fins educacionais e de monitoramento local.

5.6 Anomalias, Ajustes e Limitações

A principal dificuldade operacional ocorreu nas primeiras tentativas, quando a equipe ainda não dominava o Wokwi e tinha receio de conectar os componentes de forma inadequada. A orientação do professor Ricardo levou à adoção de testes individualizados. Posteriormente, foi identificada a necessidade de possíveis melhorias no condicionamento do LDR e do MQ-135; os componentes necessários foram disponibilizados pelo docente para utilização futura. O protótipo continua dependente de protoboard, rede Wi-Fi local, calibração do sensor de gás e estimativa aproximada de luminosidade, fatores que limitam a extrapolação para operação meteorológica de campo.

6 CONCLUSÕES E RECOMENDAÇÕES

6.1 Conclusão

O projeto resultou em um protótipo funcional de estação meteorológica baseado em ESP32 e quatro sensores ambientais. Os testes individuais confirmaram o funcionamento do DHT22, LDR, MQ-135 e BMP280, e a etapa integrada demonstrou a aquisição simultânea e a apresentação dos dados reais no dashboard por Wi-Fi. A solução também reuniu elementos de programação embarcada, comunicação HTTP, desenvolvimento web, modelagem de dados e fundamentos de circuitos e lógica. Os objetivos de montagem, leitura dos sensores e demonstração da interface foram atendidos nas condições de laboratório adotadas. As evidências obtidas não caracterizam a solução como instrumento meteorológico de precisão, mas demonstram adequadamente o funcionamento do protótipo acadêmico. Como continuidade, recomenda-se ampliar a robustez elétrica, a persistência de dados e o conjunto de visualizações, além de preparar a estação para operação externa.

6.2 Recomendações e Trabalhos Futuros

- Adicionar condicionamento elétrico adequado às entradas analógicas, conforme avaliação técnica dos módulos efetivamente utilizados.
- Substituir a protoboard por uma placa de circuito impresso e encapsular o conjunto em caixa adequada à exposição ambiental.
- Consolidar a integração entre aquisição, persistência em banco e dashboard em uma única arquitetura de produção.
- Implementar alimentação autônoma por painel solar e bateria, após dimensionamento elétrico apropriado.
- Expandir a coleta histórica e implementar recursos mais completos de visualização espacial e temporal.
- Explorar modelos de aprendizado de máquina somente após a obtenção de volume e qualidade de dados suficientes para treinamento e validação.

• Considerar uma rede com múltiplas estações para ampliar a cobertura espacial do monitoramento.

REFERÊNCIAS

ALENCAR FILHO, Edgard de. Iniciação à lógica matemática. São Paulo: Nobel, 2008.

GUADAGNINI, P. H.; ROCHA, F. S.; BARLETTE, V. E. Um medidor de luminosidade com módulo sensor integrado e aquisição automática de dados com aplicações didáticas. Revista Brasileira de Ensino de Física, v. 41, n. 4, e20180294, 2019.

IDOETA, Ivan Valeije; CAPUANO, Francisco Gabriel. Elementos de eletrônica digital. 42. ed. São Paulo: Érica, 2019.

INTERNATIONAL TELECOMMUNICATION UNION. Recommendation ITU-T Y.4000/Y.2060: Overview of the Internet of things. Geneva: International Telecommunication Union, 2012. Disponível em: https://www.itu.int/rec/T-REC-Y.4000. Acesso em: 2 out. 2026.

KOLBAN, Neil. Kolban's Book on ESP32. Leanpub, 2018. Disponível em: https://leanpub.com/kolban-ESP32. Acesso em: 2 out. 2026.

LOURENÇO, Antônio Carlos de; CHOUERI JÚNIOR, Salomão; GOMES, Sabrina Rodero Ferreira; CRUZ, Eduardo César Alves; ARAÚJO, Celso de. Circuitos digitais: estude e use. 9. ed. São Paulo: Érica, 2009.

MENEZES, Nilo Ney Coutinho. Introdução à programação com Python: algoritmos e lógica de programação para iniciantes. São Paulo: Novatec, 2019.

OLIVEIRA, Sérgio de. Internet das coisas com ESP8266, ESP32 e Raspberry Pi. São Paulo: Novatec, 2017.

SERVIÇO NACIONAL DE APRENDIZAGEM INDUSTRIAL. Fundamentos de automação. Série Eletroeletrônica. 2016.

THOMAZINI, Daniel; ALBUQUERQUE, Pedro Urbano Braga de. Sensores industriais. 9. ed. São Paulo: Érica, 2020.

WENDLING, Marcelo. Sensores. Guaratinguetá: Universidade Estadual Paulista, Colégio Técnico Industrial de Guaratinguetá, 2010.

WINTERLE, Paulo. Vetores e geometria analítica. 2. ed. São Paulo: Pearson, 2014.

APÊNDICES

Os apêndices a seguir reúnem os materiais produzidos pela própria equipe e selecionados para documentar a implementação. Os códigos são apresentados com a finalidade de permitir rastreabilidade entre a descrição e os artefatos do projeto. Credenciais de rede presentes nos arquivos de desenvolvimento foram substituídas por marcadores nesta documentação por segurança.

Apêndice A – Firmware do ESP32

Arquivo: main.py

```
import gc
import json
import math
import time
import network
import uasyncio as asyncio
from machine import Pin, ADC, I2C
from bmp280 import BMP280

# Credenciais para reconexao automatica (caso o sinal oscile)
WIFI_SSID = "<REDE_WIFI>"
WIFI_PASS = "<SENHA_WIFI>"

# ------------------------------------------------------------------------------
# 1. INICIALIZAÇÃO DE SENSORES
# ------------------------------------------------------------------------------
# BMP280 (I2C: SDA=21, SCL=22)
sensor_bmp = None
try:
    i2c = I2C(0, sda=Pin(21), scl=Pin(22), freq=100000)
    sensor_bmp = BMP280(i2c, addr=0x76)
    print("[HARDWARE] BMP280 detectado no I2C (0x76).")
except Exception as e:
    print("[AVISO] Falha ao inicializar BMP280:", e)

# DHT22 (GPIO 4)
sensor_dht = None
try:
    import dht
    sensor_dht = dht.DHT22(Pin(4))
    print("[HARDWARE] DHT22 configurado na GPIO 4.")
except Exception as e:
    print("[AVISO] Falha ao inicializar DHT22:", e)

# LDR (Sensor de Luminosidade no pino ADC 34)
adc_ldr = None
try:
    adc_ldr = ADC(Pin(34))
    adc_ldr.atten(ADC.ATTN_11DB)
    print("[HARDWARE] LDR configurado na GPIO 34.")
except Exception as e:
    print("[AVISO] Falha ao inicializar LDR:", e)

# MQ-135 (Sensor de Qualidade do Ar no pino ADC 35)
adc_mq135 = None
```

```
mq135_r0 = None

try:
    adc_mq135 = ADC(Pin(35))
    adc_mq135.atten(ADC.ATTN_11DB)
    print("[HARDWARE] MQ-135 configurado na GPIO 35.")
except Exception as e:
    print("[AVISO] Falha ao inicializar MQ-135:", e)

def ler_luminosidade():
    """Calcula iluminancia em Lux com base no LDR na GPIO 34"""
    if not adc_ldr:
        return 0.0
    try:
        raw = adc_ldr.read()
        if raw <= 15:
            return 3000.0
        v_out = (raw / 4095.0) * 3.3
        if (3.3 - v_out) <= 0.01:
            return 0.0
        r_ldr = 10.0 * v_out / (3.3 - v_out)
        lux = 500.0 * math.pow(r_ldr, -1.4)
        return round(min(lux, 5000.0), 1)
    except Exception:
        return 0.0

def ler_qualidade_ar():
    """Calcula concentracao em PPM com base no MQ-135 na GPIO 35"""
    global mq135_r0
    if not adc_mq135:
        return 400
    try:
        # Média de 8 amostras para filtrar ruído do conversor ADC
        leituras = [adc_mq135.read() for _ in range(8)]
        raw = sum(leituras) / len(leituras)

        if raw <= 30:
            print("[MQ-135] Leitura muito baixa (verifique conexão física ou pino ADC). RAW:",
round(raw))
            return 400

        v_out = (raw / 4095.0) * 3.3
        if (3.3 - v_out) <= 0.03:
            return 2500  # Saturação máxima por fumaça/gás intenso

        rs = ((3.3 - v_out) / max(0.01, v_out)) * 10.0

        # Calibra o R0 no ambiente inicial (~400 ppm baseline em ar limpo)
        if mq135_r0 is None:
```

```
            mq135_r0 = rs / 0.637
            print(f"[MQ-135] Calibração de baseline concluída: R0={mq135_r0:.2f} kOhms
(RAW={round(raw)})")

        ratio = rs / mq135_r0
        # Curva exponencial padrão do sensor MQ-135 para CO2/gases
        ppm = 110.47 * math.pow(max(0.1, ratio), -2.862)

        # Limita para uma faixa lógica e realista de exibição
        ppm_final = int(max(300, min(ppm, 3500)))
        print(f"[MQ-135] RAW: {round(raw)} | Vout: {v_out:.2f}V | Rs: {rs:.1f}k | PPM:
{ppm_final}")
        return ppm_final
    except Exception as e:
        print("[MQ-135 ERRO]", e)
        return 400

def obter_leituras():
    temp = 0.0
    pressao = 1013.25
    umid = 0.0
    status_sensor = "Online"

    # Leitura BMP280
    if sensor_bmp:
        try:
            temp_bmp, pressao_bmp = sensor_bmp.ler_valores()
            temp = temp_bmp
            pressao = pressao_bmp
        except Exception as e:
            print("[BMP280] Erro de leitura:", e)
            status_sensor = "Erro BMP280"

    # Leitura DHT22
    if sensor_dht:
        try:
            sensor_dht.measure()
            umid = round(sensor_dht.humidity(), 1)
            if not sensor_bmp:
                temp = round(sensor_dht.temperature(), 1)
        except Exception:
            pass

    return {
        "temperatura": round(temp, 1),
        "pressao": round(pressao, 2),
        "umidade": round(umid, 1),
        "qualidade_ar": ler_qualidade_ar(),
        "luminosidade": ler_luminosidade(),
```

```
        "status": status_sensor
    }

# ------------------------------------------------------------------------------
# 2. SERVIÇO DE ARQUIVOS ESTÁTICOS
# ------------------------------------------------------------------------------
async def enviar_arquivo(writer, caminho, content_type):
    try:
        with open(caminho, 'rb') as f:
            headers = (
                f"HTTP/1.1 200 OK\r\n"
                f"Content-Type: {content_type}\r\n"
                f"Cache-Control: max-age=60\r\n"
                f"Connection: close\r\n\r\n"
            )
            await writer.awrite(headers.encode('utf-8'))
            while True:
                bloco = f.read(512)
                if not bloco:
                    break
                await writer.awrite(bloco)
    except OSError:
        await writer.awrite(b"HTTP/1.1 404 Not Found\r\nContent-Type: text/plain\r\nConnection:
close\r\n\r\n404 Arquivo Nao Encontrado.")

# ------------------------------------------------------------------------------
# 3. MANIPULADOR HTTP COM SUPORTE A CORS E ROTAS
# ------------------------------------------------------------------------------
async def handle_client(reader, writer):
    try:
        linha = await asyncio.wait_for(reader.readline(), timeout=3.0)
        if not linha:
            return

        partes = linha.decode().split()
        if len(partes) < 2:
            return

        metodo = partes[0].upper()
        rota = partes[1]

        while True:
            h = await asyncio.wait_for(reader.readline(), timeout=1.0)
            if h == b"\r\n" or not h:
                break

        if metodo == "OPTIONS":
            cors_resp = (
                "HTTP/1.1 204 No Content\r\n"
```

```
                "Access-Control-Allow-Origin: *\r\n"
                "Access-Control-Allow-Methods: GET, POST, OPTIONS\r\n"
                "Access-Control-Allow-Headers: *\r\n"
                "Access-Control-Allow-Private-Network: true\r\n"
                "Connection: close\r\n\r\n"
            )
            await writer.awrite(cors_resp.encode('utf-8'))
            return

        if rota == "/api/dados":
            corpo = json.dumps(obter_leituras())
            headers = (
                "HTTP/1.1 200 OK\r\n"
                "Content-Type: application/json; charset=utf-8\r\n"
                "Access-Control-Allow-Origin: *\r\n"
                "Access-Control-Allow-Private-Network: true\r\n"
                "Connection: close\r\n\r\n"
            )
            await writer.awrite(headers.encode('utf-8') + corpo.encode('utf-8'))

        elif rota in ("/", "/index.html"):
            await enviar_arquivo(writer, "Dashboard/index.html", "text/html; charset=utf-8")
        elif rota == "/style.css":
            await enviar_arquivo(writer, "Dashboard/style.css", "text/css")
        elif rota == "/app.js":
            await enviar_arquivo(writer, "Dashboard/app.js", "application/javascript")
        else:
            await writer.awrite(b"HTTP/1.1 404 Not Found\r\nContent-Type:
text/plain\r\nConnection: close\r\n\r\nRota inexistente.")

    except asyncio.TimeoutError:
        pass
    except Exception as e:
        print("[ERRO REQUISICAO]", e)
    finally:
        try:
            await writer.drain()
            await writer.wait_closed()
        except Exception:
            pass
        gc.collect()

# ------------------------------------------------------------------------
# 4. MONITOR DE RECONEXÃO WI-FI (AUTONOMIA SEM FIO)
# ------------------------------------------------------------------------
async def monitor_wifi():
    wlan = network.WLAN(network.STA_IF)
    while True:
```

```
        await asyncio.sleep(15)
        if not wlan.isconnected():
            print("[WI-FI] Sinal oscilou. Reconectando...")
            try:
                wlan.connect(WIFI_SSID, WIFI_PASS)
            except Exception:
                pass

# ----------------------------------------------------------------------------
# 5. EXECUÇÃO DO SERVIDOR ASSÍNCRONO
# ----------------------------------------------------------------------------
async def main():
    print("[SERVIDOR] Servidor Web HTTP Assíncrono iniciado na porta 80...")
    asyncio.create_task(monitor_wifi())
    servidor = await asyncio.start_server(handle_client, "0.0.0.0", 80, backlog=3)
    while True:
        await asyncio.sleep(3600)

try:
    asyncio.run(main())
except KeyboardInterrupt:
    print("[SERVIDOR] Finalizado.")
```

Arquivo: BMP 280.py

```
import time
from machine import I2C

class BMP280:
    def __init__(self, i2c: I2C, addr: int = 0x76):
        self.i2c = i2c
        self.addr = addr

        # Verifica se o sensor está presente no barramento
        devices = self.i2c.scan()
        if self.addr not in devices:
            # Tenta o endereço alternativo 0x77
            if 0x77 in devices:
                self.addr = 0x77
            else:
                raise RuntimeError(f"BMP280 não encontrado no endereço 0x76 ou 0x77. Dispositivos
achados: {devices}")

        # Leitura dos registradores de calibração do BMP280
        self._read_calibration_data()

        # Configura o registrador de controle (ctrl_meas - 0xF4)
        # osrs_t x1 (001), osrs_p x1 (001), mode NORMAL (11) -> 0x27
        self.i2c.writeto_mem(self.addr, 0xF4, b'\x27')

        # Configura o registrador de configuração (config - 0xF5)
```

```
        # t_sb 0.5ms (000), filter off (000), spi3w_en off (0) -> 0x00
        self.i2c.writeto_mem(self.addr, 0xF5, b'\x00')
        time.sleep(0.1)

    def _read_calibration_data(self):
        # Lê 24 bytes de calibração a partir do registrador 0x88
        calib = self.i2c.readfrom_mem(self.addr, 0x88, 24)

        # Descompacta coeficientes de compensação de Temperatura
        self.dig_T1 = calib[1] << 8 | calib[0]
        self.dig_T2 = self._to_signed(calib[3] << 8 | calib[2])
        self.dig_T3 = self._to_signed(calib[5] << 8 | calib[4])

        # Descompacta coeficientes de compensação de Pressão
        self.dig_P1 = calib[7] << 8 | calib[6]
        self.dig_P2 = self._to_signed(calib[9] << 8 | calib[8])
        self.dig_P3 = self._to_signed(calib[11] << 8 | calib[10])
        self.dig_P4 = self._to_signed(calib[13] << 8 | calib[12])
        self.dig_P5 = self._to_signed(calib[15] << 8 | calib[14])
        self.dig_P6 = self._to_signed(calib[17] << 8 | calib[16])
        self.dig_P7 = self._to_signed(calib[19] << 8 | calib[18])
        self.dig_P8 = self._to_signed(calib[21] << 8 | calib[20])
        self.dig_P9 = self._to_signed(calib[23] << 8 | calib[22])

    @staticmethod
    def _to_signed(val):
        return val - 65536 if val > 32767 else val

    def ler_valores(self):
        # Lê 6 bytes a partir de 0xF7 (F7..FC: msb, lsb, xlsb da pressão e temperatura)
        data = self.i2c.readfrom_mem(self.addr, 0xF7, 6)

        adc_p = (data[0] << 12) | (data[1] << 4) | (data[2] >> 4)
        adc_t = (data[3] << 12) | (data[4] << 4) | (data[5] >> 4)

        # 1. Cálculo de compensação da Temperatura (necessário para calcular a pressão)
        var1 = (((adc_t >> 3) - (self.dig_T1 << 1)) * self.dig_T2) >> 11
        var2 = (((((adc_t >> 4) - self.dig_T1) * ((adc_t >> 4) - self.dig_T1)) >> 12) * self.dig_T3)
>> 14
        t_fine = var1 + var2
        temperatura = ((t_fine * 5 + 128) >> 8) / 100.0

        # 2. Cálculo de compensação da Pressão Atmosférica
        var1 = t_fine - 128000
        var2 = var1 * var1 * self.dig_P6
        var2 = var2 + ((var1 * self.dig_P5) << 17)
        var2 = var2 + (self.dig_P4 << 35)
        var1 = ((var1 * var1 * self.dig_P3) >> 8) + ((var1 * self.dig_P2) << 12)
        var1 = (((1 << 47) + var1) * self.dig_P1) >> 33
```

```
if var1 == 0:
    pressao_hpa = 0.0  # Evita divisão por zero
else:
    p = 1048576 - adc_p
    p = int(((p << 31) - var2) * 3125 / var1)
    var1 = (self.dig_P9 * (p >> 13) * (p >> 13)) >> 25
    var2 = (self.dig_P8 * p) >> 19
    p = ((p + var1 + var2) >> 8) + (self.dig_P7 << 4)

    # Converte Pa para hPa (divisão por 100 e depois por 256 devido à escala de ponto fixo)
    pressao_hpa = (p / 256.0) / 100.0

return round(temperatura, 1), round(pressao_hpa, 2)
```

Apêndice B – API / Backend em Python

Arquivo: python.py

```
from fastapi import FastAPI, HTTPException, status
from pydantic import BaseModel, Field
import mysql.connector
from mysql.connector import Error

app = FastAPI(title="API Estação Meteorológica")

# Configuração da Base de Dados MySQL
DB_CONFIG = {
    'host': 'localhost',
    'user': 'root',
    'password': 'sua_senha_aqui',
    'database': 'estacao_meteorologica',
    'port': 3306
}

# Modelo de Validação dos Dados de Entrada (Payload do ESP32)
class TelemetriaSchema(BaseModel):
    id_estacao: int = Field(default=1, description="ID da Estação Meteorológica")
    temperatura: float = Field(..., description="Temperatura em °C")
    umidade: float = Field(..., description="Umidade relativa em %")
    pressao: float = Field(..., description="Pressão atmosférica em hPa")
    qualidade_ar: int = Field(..., description="Qualidade do ar calculada em PPM")
    luminosidade: float = Field(..., description="Luminosidade calculada em Lux")

@app.post("/api/leituras", status_code=status.HTTP_201_CREATED)
def receber_leituras(dados: TelemetriaSchema):
    connection = None
    cursor = None
    try:
        # 1. Tenta estabelecer ligação com o MySQL
        connection = mysql.connector.connect(**DB_CONFIG)

        if not connection.is_connected():
            raise HTTPException(
                status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
                detail="Não foi possível estabelecer ligação com a base de dados."
            )

        cursor = connection.cursor()

        # 2. Comando SQL de Inserção na tabela 'leitura'
        sql_insert = """
            INSERT INTO leitura
            (id_estacao, temperatura, umidade, pressao, qualidade_ar, luminosidade, data_hora)
            VALUES (%s, %s, %s, %s, %s, %s, NOW())
```

```
        """

        valores = (
            dados.id_estacao,
            dados.temperatura,
            dados.umidade,
            dados.pressao,
            dados.qualidade_ar,
            dados.luminosidade
        )

        # 3. Executa a inserção e confirma a transação
        cursor.execute(sql_insert, valores)
        connection.commit()  # Garante a persistência física dos dados no disco

        return {
            "status": "sucesso",
            "mensagem": "Leitura gravada com sucesso na base de dados.",
            "registos_afetados": cursor.rowcount
        }

    except Error as db_err:
        # Em caso de erro de MySQL, faz o rollback para cancelar a transação
        if connection and connection.is_connected():
            connection.rollback()

        print(f"[ERRO MYSQL] Falha na persistência: {db_err}")

        # Retorna o código HTTP 500 informando o erro de persistência
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"Erro ao persistir os dados no MySQL: {str(db_err)}"
        )

    except Exception as e:
        if connection and connection.is_connected():
            connection.rollback()

        print(f"[ERRO INESPERADO] {e}")

        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail="Ocorreu um erro interno no servidor ao processar a requisição."
        )

    finally:
        # 4. Garante que os recursos da base de dados são libertados com segurança
        if cursor:
            cursor.close()
        if connection and connection.is_connected():
```

```
        connection.close()
from datetime import datetime

@app.get("/api/leituras/ultimas-24h")
def obter_ultimas_leituras():
    connection = None
    cursor = None
    try:
        connection = mysql.connector.connect(**DB_CONFIG)
        cursor = connection.cursor(dictionary=True)

        # Consulta os últimos registros do banco
        cursor.execute("""
            SELECT id, id_estacao, temperatura, umidade, pressao,
                qualidade_ar, luminosidade, data_hora
            FROM leitura
            ORDER BY data_hora DESC
            LIMIT 50
        """)
        historico = cursor.fetchall()

        esp32_ativo = False
        if historico:
            ultima_leitura_time = historico[0]['data_hora']
            # Calcula a diferença de tempo em segundos entre o momento atual e a última leitura
            tempo_decorrido = (datetime.now() - ultima_leitura_time).total_seconds()

            # Se a última leitura foi há menos de 20 segundos, o ESP32 é considerado ONLINE
            if tempo_decorrido <= 20:
                esp32_ativo = True

        # Agrega estatísticas das últimas 24h
        cursor.execute("""
            SELECT
                COALESCE(MAX(temperatura), 0) as temp_max,
                COALESCE(MIN(temperatura), 0) as temp_min,
                COALESCE(ROUND(AVG(umidade), 1), 0) as umid_media,
                COALESCE(MAX(qualidade_ar), 0) as air_max
            FROM leitura
            WHERE data_hora >= NOW() - INTERVAL 1 DAY
        """)
        stats = cursor.fetchone()

        return {
            "esp32_ativo": esp32_ativo,
            "historico": historico,
            "estatisticas_24h": stats
        }
```

```
except Exception as e:
    raise HTTPException(status_code=500, detail=str(e))
finally:
    if cursor: cursor.close()
    if connection: connection.close()
```

Apêndice C – Dashboard Web

Arquivo: index.html

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Estação IoT ESP32 — BMP280</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <div class="dashboard-wrapper">

    <!-- Topbar Header -->
    <header class="main-header">
      <div class="brand">
        <div class="brand-icon">🌤️</div>
        <div>
          <h1>Estação Meteorológica Inteligente</h1>
          <p>ESP32 MicroPython • Firebase Cloud Hosting</p>
        </div>
      </div>
      <div class="header-right">
        <span class="clock-badge" id="clock">--:--:--</span>
        <div class="status-badge" id="statusBadge">
          <span class="status-dot" id="statusDot"></span>
          <span id="statusText">Conectando...</span>
        </div>
      </div>
    </header>

    <!-- Grid de Métricas Principais -->
    <main class="metrics-grid">

      <!-- Card Temperatura -->
      <article class="metric-card card-temp">
        <div class="card-icon-wrapper temp-bg">🌡️</div>
        <div class="card-details">
          <span class="card-label">Temperatura</span>
          <div class="card-value-group">
            <span class="card-value" id="valTemp">--</span>
            <span class="card-unit">°C</span>
          </div>
          <span class="card-status-pill" id="stTemp">Aguardando dados...</span>
        </div>
      </article>
```

```
<!-- Card Pressão Atmosférica -->
<article class="metric-card card-press">
  <div class="card-icon-wrapper press-bg">📊</div>
  <div class="card-details">
    <span class="card-label">Pressão Atmosférica</span>
    <div class="card-value-group">
      <span class="card-value" id="valPress">--</span>
      <span class="card-unit">hPa</span>
    </div>
    <span class="card-status-pill" id="stPress">Aguardando dados...</span>
  </div>
</article>

<!-- Card Umidade -->
<article class="metric-card card-humid">
  <div class="card-icon-wrapper humid-bg">💧</div>
  <div class="card-details">
    <span class="card-label">Umidade Relativa</span>
    <div class="card-value-group">
      <span class="card-value" id="valUmid">--</span>
      <span class="card-unit">%</span>
    </div>
    <span class="card-status-pill" id="stUmid">Aguardando dados...</span>
  </div>
</article>

<!-- Card Qualidade do Ar -->
<article class="metric-card card-air">
  <div class="card-icon-wrapper air-bg">🍃</div>
  <div class="card-details">
    <span class="card-label">Qualidade do Ar</span>
    <div class="card-value-group">
      <span class="card-value" id="valAir">--</span>
      <span class="card-unit">PPM</span>
    </div>
    <span class="card-status-pill" id="stAir">Aguardando dados...</span>
  </div>
</article>

<!-- Card Luminosidade -->
<article class="metric-card card-light">
  <div class="card-icon-wrapper light-bg">☀️</div>
  <div class="card-details">
    <span class="card-label">Luminosidade</span>
    <div class="card-value-group">
      <span class="card-value" id="valLight">--</span>
      <span class="card-unit">Lux</span>
    </div>
```

```
        <span class="card-status-pill" id="stLight">Aguardando dados...</span>
      </div>
    </article>

  </main>

  <!-- Painel de Diagnóstico do Sistema -->
  <section class="info-panel">
    <h3>⚡ Diagnóstico de Hardware e Rede</h3>
    <div class="diagnostics-grid">
      <div class="diag-item">
        <span>Sensor Barométrico</span>
        <strong>GY-BMP280 (I2C: 0x76)</strong>
      </div>
      <div class="diag-item">
        <span>Pinos de Comunicação</span>
        <strong>SDA (GPIO 21) • SCL (GPIO 22)</strong>
      </div>
      <div class="diag-item">
        <span>Sensor Higrométrico</span>
        <strong>DHT22 (GPIO 4)</strong>
      </div>
      <div class="diag-item">
        <span>Cloud Provider</span>
        <strong>Google Firebase Hosting</strong>
      </div>
      <div class="diag-item">
        <span>Última Sincronização</span>
        <strong id="lastSync">--:--:--</strong>
      </div>
      <div class="diag-item">
        <span>Estado da API</span>
        <strong id="apiStatus">Pronta</strong>
      </div>
    </div>
  </section>

  <footer class="main-footer">
    <p>Estação Meteorológica ESP32 • MicroPython v1.20+ • Google Firebase</p>
  </footer>

 </div>

 <script type="module" src="app.js"></script>
</body>
</html>
```

Arquivo: style.css

```
:root {
```

```
  --bg-color: #0b1120;
  --panel-bg: rgba(30, 41, 59, 0.7);
  --panel-border: rgba(255, 255, 255, 0.08);
  --text-main: #f8fafc;
  --text-muted: #94a3b8;
  --accent-temp: #f43f5e;
  --accent-press: #38bdf8;
  --accent-humid: #0284c7;
  --accent-air: #10b981;
  --accent-light: #eab308;
  --status-online: #10b981;
  --status-offline: #f43f5e;
}

* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
  font-family: system-ui, -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, sans-serif;
}

body {
  background-color: var(--bg-color);
  background-image:
    radial-gradient(at 0% 0%, rgba(56, 189, 248, 0.1) 0px, transparent 50%),
    radial-gradient(at 100% 100%, rgba(244, 63, 94, 0.08) 0px, transparent 50%);
  color: var(--text-main);
  min-height: 100vh;
  padding: 1.5rem;
  display: flex;
  justify-content: center;
}

.dashboard-wrapper {
  width: 100%;
  max-width: 1150px;
}

/* Header */
.main-header {
  background: var(--panel-bg);
  backdrop-filter: blur(12px);
  border: 1px solid var(--panel-border);
  border-radius: 16px;
  padding: 1.25rem 1.75rem;
  display: flex;
  justify-content: space-between;
  align-items: center;
```

```
  flex-wrap: wrap;
  gap: 1rem;
  margin-bottom: 1.5rem;
}

.brand {
  display: flex;
  align-items: center;
  gap: 1rem;
}

.brand-icon {
  font-size: 2.2rem;
}

.brand h1 {
  font-size: 1.35rem;
  font-weight: 700;
  letter-spacing: -0.02em;
}

.brand p {
  font-size: 0.82rem;
  color: var(--text-muted);
}

.header-right {
  display: flex;
  align-items: center;
  gap: 0.85rem;
}

.clock-badge {
  background: rgba(255, 255, 255, 0.05);
  border: 1px solid var(--panel-border);
  padding: 0.4rem 0.85rem;
  border-radius: 9999px;
  font-size: 0.85rem;
  font-family: monospace;
}

.status-badge {
  display: flex;
  align-items: center;
  gap: 0.5rem;
  background: rgba(255, 255, 255, 0.05);
  border: 1px solid var(--panel-border);
  padding: 0.4rem 0.9rem;
  border-radius: 9999px;
```

```
  font-size: 0.85rem;
  font-weight: 500;
}

.status-dot {
  width: 9px;
  height: 9px;
  border-radius: 50%;
  background-color: var(--status-offline);
  transition: background-color 0.3s ease, box-shadow 0.3s ease;
}

.status-badge.online .status-dot {
  background-color: var(--status-online);
  box-shadow: 0 0 10px var(--status-online);
}

/* Grid de Métricas */
.metrics-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(210px, 1fr));
  gap: 1.25rem;
  margin-bottom: 1.5rem;
}

.metric-card {
  background: var(--panel-bg);
  backdrop-filter: blur(12px);
  border: 1px solid var(--panel-border);
  border-radius: 16px;
  padding: 1.4rem;
  display: flex;
  align-items: flex-start;
  gap: 1rem;
  transition: transform 0.2s ease, border-color 0.2s ease;
}

.metric-card:hover {
  transform: translateY(-2px);
  border-color: rgba(255, 255, 255, 0.16);
}

.card-icon-wrapper {
  width: 48px;
  height: 48px;
  border-radius: 12px;
  display: flex;
  align-items: center;
  justify-content: center;
```

```
  font-size: 1.35rem;
  flex-shrink: 0;
}

.temp-bg { background: rgba(244, 63, 94, 0.15); color: var(--accent-temp); }
.press-bg { background: rgba(56, 189, 248, 0.15); color: var(--accent-press); }
.humid-bg { background: rgba(2, 132, 199, 0.15); color: var(--accent-humid); }
.air-bg { background: rgba(16, 185, 129, 0.15); color: var(--accent-air); }
.light-bg { background: rgba(234, 179, 8, 0.15); color: var(--accent-light); }

.card-details {
  display: flex;
  flex-direction: column;
  flex-grow: 1;
}

.card-label {
  font-size: 0.78rem;
  font-weight: 500;
  color: var(--text-muted);
  text-transform: uppercase;
  letter-spacing: 0.04em;
}

.card-value-group {
  display: flex;
  align-items: baseline;
  gap: 0.3rem;
  margin: 0.25rem 0 0.5rem 0;
}

.card-value {
  font-size: 1.85rem;
  font-weight: 700;
  letter-spacing: -0.02em;
}

.card-unit {
  font-size: 0.85rem;
  color: var(--text-muted);
  font-weight: 400;
}

.card-status-pill {
  font-size: 0.75rem;
  color: var(--text-muted);
  font-weight: 500;
}
```

```
/* Painel de Diagnóstico */
.info-panel {
  background: var(--panel-bg);
  backdrop-filter: blur(12px);
  border: 1px solid var(--panel-border);
  border-radius: 16px;
  padding: 1.5rem;
  margin-bottom: 1.5rem;
}

.info-panel h3 {
  font-size: 1rem;
  margin-bottom: 1.1rem;
  color: var(--text-main);
}

.diagnostics-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
  gap: 1rem;
}

.diag-item {
  background: rgba(255, 255, 255, 0.03);
  border: 1px solid rgba(255, 255, 255, 0.04);
  padding: 0.85rem 1rem;
  border-radius: 10px;
  display: flex;
  flex-direction: column;
  gap: 0.25rem;
}

.diag-item span {
  font-size: 0.75rem;
  color: var(--text-muted);
}

.diag-item strong {
  font-size: 0.88rem;
  color: var(--text-main);
  font-weight: 600;
}

/* Rodapé */
.main-footer {
  text-align: center;
  padding: 1rem;
  font-size: 0.78rem;
  color: var(--text-muted);
```

```
}
```

Arquivo: app.js

```
// Importa SDKs oficiais do Firebase via CDN moderna
import { initializeApp } from "https://www.gstatic.com/firebasejs/11.0.1/firebase-app.js";
import { getAnalytics } from "https://www.gstatic.com/firebasejs/11.0.1/firebase-analytics.js";

// Configuração do Firebase fornecida
const firebaseConfig = {
  apiKey: "AIzaSyCBTm39yjUExp25pzcVe0BaIS_qOAo5YFI",
  authDomain: "estacao-metedologica.firebaseapp.com",
  projectId: "estacao-metedologica",
  storageBucket: "estacao-metedologica.firebasestorage.app",
  messagingSenderId: "702509499670",
  appId: "1:702509499670:web:ca9a7049522545de7ffc37",
  measurementId: "G-LLM0T0Y1N0"
};

// Inicializa Firebase
const app = initializeApp(firebaseConfig);
try {
  getAnalytics(app);
} catch (e) {
  console.log("[FIREBASE] Analytics operando em modo local.");
}

// Relógio em tempo real
setInterval(() => {
  const clockEl = document.getElementById('clock');
  if (clockEl) {
    clockEl.innerText = new Date().toLocaleTimeString('pt-BR');
  }
}, 1000);

// Endpoint inteligente para consumo
const ESP32_IP = '192.168.204.112';
const API_URL = (window.location.protocol.startsWith('http') && window.location.hostname !
== 'localhost' && !window.location.hostname.includes('web.app') && !
window.location.hostname.includes('firebaseapp.com'))
  ? '/api/dados'
  : `http://${ESP32_IP}/api/dados`;

async function atualizarDashboard() {
  const statusBadge = document.getElementById('statusBadge');
  const statusText = document.getElementById('statusText');
  const apiStatus = document.getElementById('apiStatus');
  const lastSync = document.getElementById('lastSync');

  try {
```

```
const controller = new AbortController();
const timeoutId = setTimeout(() => controller.abort(), 3500);

const response = await fetch(API_URL, { signal: controller.signal });
clearTimeout(timeoutId);

if (!response.ok) {
  throw new Error(`HTTP ${response.status}`);
}

const data = await response.json();

// 1. Status Geral de Conexão
statusBadge.classList.add('online');
statusText.innerText = `Online (${data.status || 'OK'})`;
apiStatus.innerText = 'Respondendo (200 OK)';
apiStatus.style.color = '#10b981';
lastSync.innerText = new Date().toLocaleTimeString('pt-BR');

// 2. Atualizar Temperatura
const temp = parseFloat(data.temperatura) || 0.0;
document.getElementById('valTemp').innerText = temp.toFixed(1);
const stTemp = document.getElementById('stTemp');
if (temp > 30) {
  stTemp.innerText = 'Temperatura Elevada';
  stTemp.style.color = '#f43f5e';
} else if (temp < 18) {
  stTemp.innerText = 'Temperatura Baixa';
  stTemp.style.color = '#38bdf8';
} else {
  stTemp.innerText = 'Faixa Confortável';
  stTemp.style.color = '#10b981';
}

// 3. Atualizar Pressão Barométrica
const press = parseFloat(data.pressao) || 0.0;
document.getElementById('valPress').innerText = press.toFixed(1);
const stPress = document.getElementById('stPress');
stPress.innerText = press > 1000 ? 'Barômetro Estável' : 'Pressão Altitude';
stPress.style.color = '#38bdf8';

// 4. Atualizar Umidade Relativa
const umid = parseFloat(data.umidade) || 0.0;
document.getElementById('valUmid').innerText = umid.toFixed(1);
const stUmid = document.getElementById('stUmid');
if (umid >= 30 && umid <= 70) {
  stUmid.innerText = 'Umidade Ideal';
  stUmid.style.color = '#10b981';
} else if (umid < 30) {
```

```
      stUmid.innerText = 'Ar Muito Seco';
      stUmid.style.color = '#eab308';
    } else {
      stUmid.innerText = 'Alta Concentração';
      stUmid.style.color = '#0284c7';
    }

    // 5. Atualizar Qualidade do Ar (Escala em PPM para CO2/Gases em ambiente)
    const air = parseInt(data.qualidade_ar) || 400;
    document.getElementById('valAir').innerText = air;
    const stAir = document.getElementById('stAir');
    if (air <= 700) {
      stAir.innerText = 'Qualidade Excelente';
      stAir.style.color = '#10b981';
    } else if (air <= 1000) {
      stAir.innerText = 'Qualidade Moderada';
      stAir.style.color = '#38bdf8';
    } else if (air <= 1500) {
      stAir.innerText = 'Ar Viciado (Ventilar)';
      stAir.style.color = '#eab308';
    } else {
      stAir.innerText = 'Alerta de Poluição';
      stAir.style.color = '#f43f5e';
    }

    // 6. Atualizar Luminosidade
    const light = parseFloat(data.luminosidade) || 0.0;
    document.getElementById('valLight').innerText = light.toFixed(1);
    const stLight = document.getElementById('stLight');
    stLight.innerText = light > 500 ? 'Luz Natural / Alta' : (light > 50 ? 'Ambiente Iluminado' :
'Penumbra / Noite');
    stLight.style.color = '#eab308';

  } catch (error) {
    statusBadge.classList.remove('online');
    statusText.innerText = 'Aguardando ESP32';
    apiStatus.innerText = window.location.protocol === 'https:'
      ? 'Aviso: HTTPS bloqueia IP local direto (Mixed Content)'
      : 'Sem resposta do ESP32';
    apiStatus.style.color = '#f43f5e';
    console.warn('[DASHBOARD] Aviso de comunicação:', error.message);
  }
}

// Inicia polling a cada 2 segundos
document.addEventListener('DOMContentLoaded', () => {
  atualizarDashboard();
  setInterval(atualizarDashboard, 2000);
```

```
});
```

Apêndice D – Script SQL do Banco de Dados

Arquivo: sql

```
-- Remove a tabela antiga caso exista para atualizar a estrutura
DROP TABLE IF EXISTS leitura;
DROP TABLE IF EXISTS estacao;

-- Tabela Estação (Mapeamento Geográfico)
CREATE TABLE estacao (
    id_estacao INT AUTO_INCREMENT PRIMARY KEY,
    latitude FLOAT NOT NULL,
    longitude FLOAT NOT NULL,
    descricao VARCHAR(100)
);

-- Inserção da estação padrão (São Paulo)
INSERT INTO estacao (id_estacao, latitude, longitude, descricao)
VALUES (1, -23.55052, -46.63330, 'Estação Principal ESP32 - SP');

-- Tabela Leitura (Persistência da Telemetria sem Pluviometria)
CREATE TABLE leitura (
    id_leitura BIGINT AUTO_INCREMENT PRIMARY KEY,
    id_estacao INT NOT NULL,
    temperatura FLOAT NOT NULL,
    umidade FLOAT NOT NULL,
    pressao FLOAT NOT NULL,
    qualidade_ar INT NOT NULL,
    luminosidade INT NOT NULL,
    data_hora TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_estacao FOREIGN KEY (id_estacao) REFERENCES estacao(id_estacao)
);
```

Apêndice E – Tabela de Custos (BOM)

Tabela 10 – Tabela de custos (BOM).

Componente	Custo
DHT22	R$ 37,00
LDR	R$ 11,49
MQ-135	R$ 25,00
BMP280	R$ 15,00
ESP32 DEVKIT V1	R$ 40,00
Kit protoboard 830 + fonte + regulador + 40 jumpers	R$ 55,00
Cabo jumper 20 cm, 40 fios, M/M, M/F e F/F	R$ 17,00
TOTAL	R$ 200,49

Fonte: elaborado pelos autores (2026).

O custo total informado pela equipe foi de R$ 200,49 para os itens relacionados. Considerando quatro integrantes, o valor médio correspondente é aproximadamente R$ 50,12 por integrante.

DECLARAÇÃO DE USO DE INTELIGÊNCIA ARTIFICIAL

Houve utilização de ferramenta de Inteligência Artificial Generativa durante a elaboração da documentação deste projeto. O uso ocorreu de forma assistiva e supervisionada, permanecendo a equipe humana responsável pela seleção das informações, conferência dos materiais do projeto, validação dos resultados, revisão do texto e decisão final sobre o conteúdo submetido.

1. IDENTIFICAÇÃO

Campo	Informação
Título do trabalho	Estação Meteorológica com ESP32
Autores	Matheus Bifaroni Carvalho; Francisco Il Guk Fermone Castro Souza; Gustavo Pereira Barreto; Lucas Vicente Pereira Silva
Curso / Instituição	Ciência da Computação – Universidade Paulista (UNIP)
Docente responsável	Marco Gomes
RA / CPF	Não informados neste registro; preencher caso sejam exigidos institucionalmente.

2. NATUREZA DO USO E DISCRIMINAÇÃO FUNCIONAL

A ferramenta foi utilizada nas seguintes categorias:

- Apoio linguístico e sintático: revisão, reescrita, clareza, organização textual e adequação da redação acadêmica.
- Codificação e métodos computacionais: leitura, comparação e análise dos scripts fornecidos pela equipe, inclusive comparação entre versões e identificação de inconsistências. A IA deste processo não gerou o código do projeto; os códigos foram produzidos pelos integrantes responsáveis pelo software.
- Tratamento e organização de dados: consolidação de informações, registros, testes, evidências audiovisuais e tabelas para a documentação.

• Prospecção bibliográfica: apoio à organização de fontes e estruturação de referências utilizadas na fundamentação.

• Estruturação e preparação de documentos: montagem, formatação e revisão dos arquivos de documentação em Word e PDF.

• Análise de consistência técnica: confronto entre o roteiro docente, materiais da disciplina, códigos e informações fornecidas pela equipe para identificação de divergências e pendências.

• Organização de figuras, tabelas e descrições técnicas para apresentação dos resultados.

A interpretação final dos resultados experimentais, a seleção definitiva das evidências e as conclusões submetidas foram realizadas e validadas pelos autores humanos.

3. ESPECIFICAÇÃO TÉCNICA E RASTREABILIDADE DO MODELO

Campo	**Informação**
Ferramenta / desenvolvedor	ChatGPT / OpenAI
Modelo / checkpoint	GPT-5.6 Luna
Ambiente de acesso	Interface web do ChatGPT
Janela temporal das interações	26/09/2026 a 03/10/2026

4. MATRIZ DE EMPREGO TÉCNICO

Categoria / seção	**Finalidade específica**	**Artefato afetado**	**Intervenção humana pós-geração**
Documentação	Organização, redação preliminar e revisão	Texto, estrutura e tabelas do relatório	100% – revisão integral pelos autores
Metodologia e desenvolvimento	Confronto entre roteiro, fatos e artefatos	Cronologia, requisitos e descrições técnicas	100% – conferência pelos autores
Software / apêndices	Leitura, comparação e identificação de inconsistências	Firmware, API, dashboard e SQL	100% – seleção e validação dos arquivos finais pelos autores
Resultados / testes	Organização dos registros e evidências	Tabelas e descrições de testes	100% – resultados conferidos pela equipe
Referências /	Apoio à organização	Referências e citações	100% – conferência

Categoria / seção	Finalidade específica	Artefato afetado	Intervenção humana pós-geração
fundamentação	bibliográfica		manual pelos autores

5. PROTOCOLO DE AUDITORIA E PROMPTS

As interações utilizadas para a elaboração desta documentação permanecem registradas no histórico da conversa em que o trabalho foi desenvolvido. Os autores declaram possuir acesso a esses prompts e às saídas correspondentes e comprometem-se a disponibilizá-los caso sejam solicitados no contexto acadêmico aplicável.

O processo de refinamento ocorreu de forma iterativa, por sucessivas rodadas de fornecimento de contexto, correção de informações, análise dos arquivos enviados e confronto com o material da disciplina. Não foi utilizado procedimento formal de Few-shot prompting nem RAG com bases proprietárias.

6. RESPONSABILIDADE ÉTICA E RIGOR FACTUAL

- Nenhuma ferramenta de Inteligência Artificial foi indicada como autora ou coautora da obra.
- Os autores mantiveram responsabilidade integral pela conferência do conteúdo técnico, dos resultados experimentais, das referências, das equações e das informações utilizadas na versão final.
- Os autores informam que realizaram a verificação manual das referências, dados numéricos e demais informações relevantes antes da submissão.
- Não foram inseridos nas ferramentas de IA dados pessoais sensíveis ou informações submetidas a sigilo ético, conforme verificação dos autores.
- Os autores assumem responsabilidade pelo conteúdo integral da obra submetida.

7. OBSERVAÇÃO SOBRE A CONFORMIDADE DO USO

A presente declaração descreve o uso efetivamente realizado durante a elaboração do trabalho. A ferramenta foi empregada como apoio técnico e textual, sob revisão e decisão dos

autores humanos. As conclusões e a responsabilidade final pelo conteúdo são dos autores. A declaração não atribui à ferramenta autoria ou responsabilidade pelo trabalho.

8. ASSINATURAS

Local: São Paulo – SP

Data: 03 / 10 / 2026

_____

Matheus Bifaroni Carvalho

_____

Francisco Il Guk Fermone Castro Souza

_____

Gustavo Pereira Barreto

_____

Lucas Vicente Pereira Silva

Ciente e de acordo: _____

Marco Gomes – Docente responsável